Tecnológico de Estudios Superiores de Ecatepec

División de Ingeniería en Sistemas Computacionales

ACTIVIDAD 7

Estructuras de control en JavaScript

Institución	Tecnológico de Estudios Superiores de Ecatepec
Carrera	División de Ingeniería en Sistemas Computacionales
Materia	Programación Web
Nombre	Marcos Joaquín García Villanueva
Nombre del Profesor	Alejandra González Sánchez
Grupo	5501
Actividad	7 — Estructuras de control en JavaScript

Introducción

Las estructuras de control son el mecanismo que permite a un programa tomar decisiones y repetir acciones. Sin ellas, el código se ejecutaría siempre de la misma forma, de arriba hacia abajo, sin capacidad de reaccionar a los datos. En JavaScript, estas estructuras son la base de cualquier aplicación web dinámica: validar un formulario, filtrar resultados o generar listas depende por completo de ellas.

En el contexto de PropertyStream Tech, las estructuras de control hacen posibles funcionalidades reales de Smart Listings y Green Properties Index: validar los datos de entrada de una propiedad (precio, ubicación, tipo) con estructuras de decisión, y generar dinámicamente listas de propiedades eco-friendly con estructuras de iteración. Este informe investiga las estructuras de control de JavaScript, las aplica en scripts funcionales, simula su integración con Firebase y documenta el uso de cada estructura.

Objetivos

- Investigar las estructuras de control de JavaScript (if, switch, for, while, do-while) y sus usos.
- Diseñar un script que valide datos de un formulario con `if` y `switch`.
- Crear un script que genere dinámicamente una lista de propiedades eco-friendly con `for` o `while`.
- Integrar los scripts con Firebase (simulación) para la interacción con una base de datos.
- Subir el código a GitHub y documentar el uso de las estructuras de control en cada script.

Estructuras de control en JavaScript

Se dividen en dos grandes grupos: las de **decisión**, que eligen qué código ejecutar según una condición, y las de **iteración** (ciclos), que repiten un bloque de código.

Estructura	Tipo	Uso típico
if / else if / else	Decisión	Elegir entre varias condiciones
switch	Decisión	Comparar una variable contra varios valores fijos
for	Iteración	Repetir un número conocido de veces o recorrer listas
while	Iteración	Repetir mientras una condición sea verdadera

Estructura	Tipo	Uso típico
do-while	Iteración	Igual que while, pero se ejecuta al menos una vez

Decisión: if y switch

```
// if / else if / else
if (precio < 8000) {
  categoria = "Económica";
} else if (precio <= 15000) {
  categoria = "Media";
} else {
  categoria = "Premium";
}

// switch
switch (tipo) {
  case "renta": descripcion = "en renta"; break;
  case "venta": descripcion = "en venta"; break;
  default:      descripcion = "no reconocido";
}
```

Iteración: for, while y do-while

```
// for: cuando se conoce el número de repeticiones
for (let i = 0; i < lista.length; i++) { /* ... */ }

// while: se repite mientras la condición sea verdadera
let j = 0;
while (j < lista.length) { j++; }

// do-while: se ejecuta al menos una vez
let k = 0;
do { k++; } while (k < 3);
```

Script 1: validación de formulario (if + switch)

El primer script valida los datos de un formulario de publicación de propiedades. Usa `if` para comprobar que los campos no estén vacíos y que el precio sea un número válido, y `switch` para clasificar el tipo de propiedad.

```
function validarFormulario(evento) {
  evento.preventDefault();
  const precio = Number(document.getElementById("precio").value);
  const ubicacion = document.getElementById("ubicacion").value;
  const tipo = document.getElementById("tipo").value;

  // Validación con if / else if
  if (ubicacion === "" || tipo === "") {
    salida.textContent = "Error: campos obligatorios.";
    return false;
  } else if (isNaN(precio)) {
    salida.textContent = "Error: el precio debe ser un número.";
    return false;
  } else if (precio <= 0) {
    salida.textContent = "Error: el precio debe ser mayor a cero.";
    return false;
  }

  // Clasificación con switch
  switch (tipo) {
    case "renta": descripcion = "propiedad en renta"; break;
    case "venta": descripcion = "propiedad en venta"; break;
    case "eco":   descripcion = "propiedad eco-friendly"; break;
    default:      descripcion = "tipo no reconocido";
  }
}
```

Uso de las estructuras: los `if` encadenados detienen el proceso y muestran un error si algún dato es inválido (campo vacío, precio no numérico o precio no positivo). El `switch` compara el valor de `tipo` contra varios casos y asigna la descripción correspondiente; el `default` cubre cualquier valor inesperado.

Script 2: lista eco-friendly dinámica (for)

El segundo script genera dinámicamente una lista de propiedades eco-friendly recorriendo un arreglo con un ciclo `for` y creando un elemento de lista por cada propiedad.

```
function generarEcoFriendly() {
  const propiedades = [
    { nombre: "Casa Solar Norte", ahorro: 40 },
    { nombre: "Depto Verde Centro", ahorro: 30 },
    { nombre: "Loft Sustentable", ahorro: 25 },
```

```
    { nombre: "Residencia EcoSur", ahorro: 55 }
  ];
  const lista = document.getElementById("listaEco");
  lista.innerHTML = "";

  for (let i = 0; i < propiedades.length; i++) {
    const p = propiedades[i];
    const item = document.createElement("li");
    item.textContent = `${p.nombre} - ahorro ${p.ahorro}%`;
    lista.appendChild(item);
  }
}
```

Uso de las estructuras: el ciclo `for` se repite tantas veces como elementos tenga el arreglo (`propiedades.length`). En cada vuelta toma una propiedad, crea un `<li>` con su nombre y ahorro energético, y lo agrega a la lista. Así la interfaz se construye de forma automática a partir de los datos.

Script 3: integración con Firebase (do-while)

El tercer script simula el guardado de una propiedad en Firebase. Usa un ciclo `do-while` para representar reintentos de conexión y `setTimeout` para imitar la latencia de red.

```
function guardarEnFirebase() {
  let intentos = 0;
  let conectado = false;

  // do-while: intenta conectar al menos una vez
  do {
    intentos++;
    if (intentos >= 2) { conectado = true; }
  } while (!conectado && intentos < 3);

  setTimeout(function () {
    if (conectado) {
      salida.textContent = `Guardado tras ${intentos} intento(s).`;
    } else {
      salida.textContent = "No se pudo conectar con Firebase.";
    }
  }, 1000);
}
```

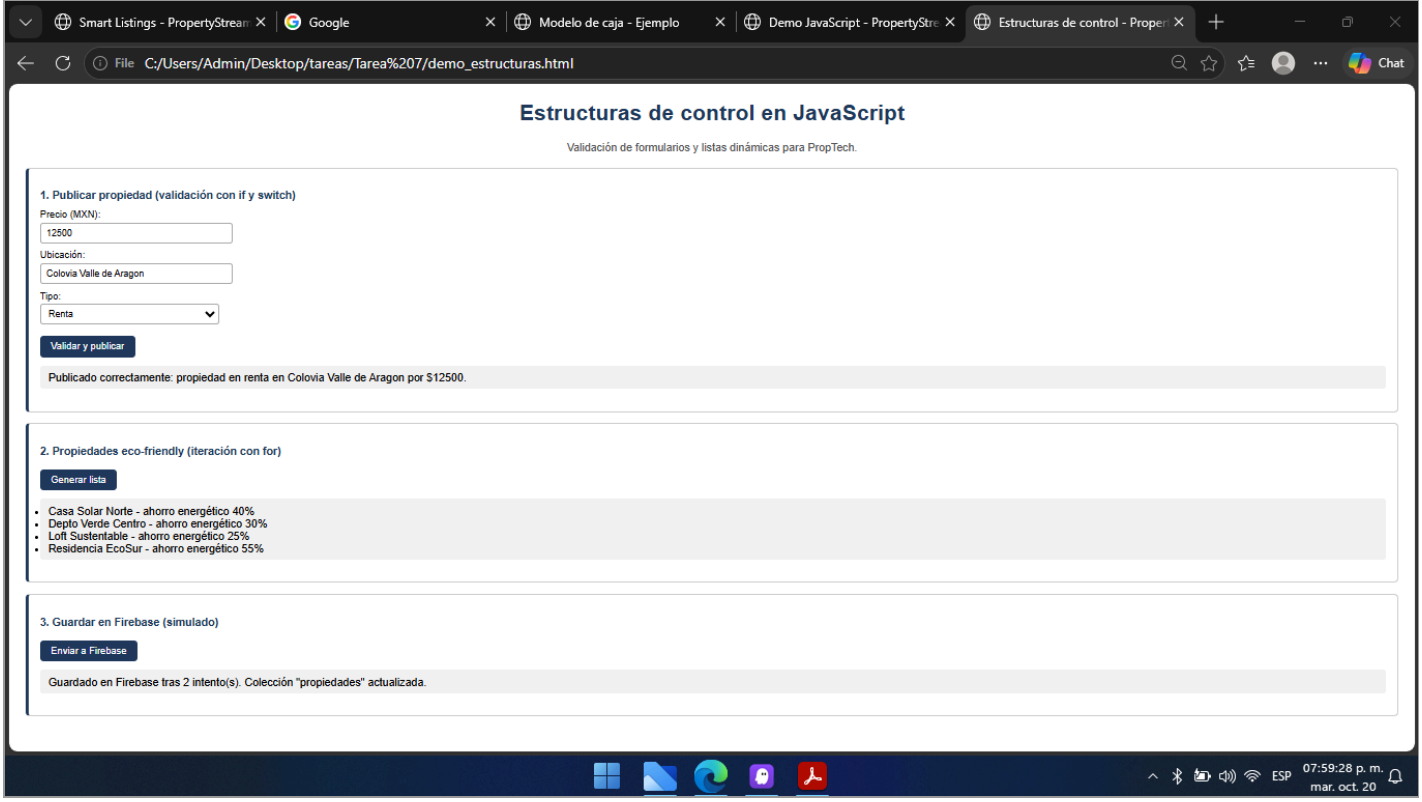
Uso de las estructuras: el `do-while` ejecuta el intento de conexión al menos una vez y lo repite mientras no haya conexión y no se superen 3 intentos. Luego, el `if/else` dentro de `setTimeout` decide el mensaje final según el resultado. Esto ilustra cómo las estructuras de control gestionan la comunicación con una base de datos real.

Relación con Smart Listings y Green Properties Index

Estos scripts reflejan funcionalidades reales de la empresa. La validación con `if` y `switch` es la que impide publicar propiedades con datos incorrectos en **Smart Listings**, protegiendo la calidad de la información. La generación dinámica con `for` es la base de **Green Properties Index**, que lista y clasifica propiedades eco-friendly según su ahorro energético. La integración con Firebase mediante `do-while` y asincronía permite que esos datos se guarden y actualicen en tiempo real.

Evidencia: scripts en ejecución

A continuación se muestra la captura de la página de demostración, donde se ejecutan los tres scripts (validación del formulario, lista eco-friendly y guardado simulado en Firebase), a partir de los archivos `demo_estructuras.html` y `demo_script.js`.



Conclusión

Las estructuras de control son esenciales para construir aplicaciones web dinámicas. Las de decisión (`if`, `switch`) permiten validar y clasificar datos, mientras que las de iteración (`for`, `while`, `do-while`) permiten procesar listas y gestionar reintentos. Aplicadas al sector PropTech, hacen posibles funcionalidades clave de Smart Listings y Green Properties Index, y combinadas con Firebase permiten que la información se valide, genere y almacene de forma confiable. El código se documentó y se subió a GitHub según lo solicitado.

Referencias

- Mozilla Developer Network. *Control flow and error handling*. MDN Web Docs.
- Mozilla Developer Network. *Loops and iteration*. MDN Web Docs.
- Google. *Firebase Documentation*. firebase.google.com.
- GitHub. *GitHub Docs*. docs.github.com.
- W3Schools. *JavaScript Switch, Loops*. w3schools.com.